Module 03: Data Preprocessing Techniques

Lecturer: Abdan Hafidz
Topic: From Raw Data to Model-Ready Inputs

---

1 Introduction: The Art of Data Preparation

Imagine you are a master chef about to prepare a gourmet meal. You wouldn't throw unwashed vegetables, whole unpeeled potatoes, and raw meat directly into a pot and hope for the best. Instead, you wash, peel, chop, and season your ingredients.

Data Preprocessing is exactly this culinary preparation for Machine Learning. Raw data is inherently messy—it is riddled with missing values, corrupted by outliers, and confusingly scaled (comparing age in years to income in millions). If we feed this "raw ingredients" directly into a mathematical model, the result will be inevitable failure. This is the essence of the **Garbage In, Garbage Out (GIGO)** principle.

In this module, we will explore the rigorous mathematical and procedural techniques required to transform raw, chaotic data into a pristine, numerical format that allows algorithms to learn effectively.

2 Handling Missing Values: The Art of Imputation

In an ideal world, every dataset is complete. In reality, data is full of holes. Sensors fail, users skip survey questions, and databases get corrupted. These gaps are represented as `NaN` (Not a Number). Most machine learning algorithms cannot handle NaNs—they rely on complete vector operations. Therefore, we must fill these gaps, a process known as **Imputation**.

The decision of *how* to impute is not trivial; it requires an understanding of the data's distribution and the nature of the missingness.

2.1 Univariate Imputation Strategies

The simplest form of imputation looks only at the feature column itself. We replace the missing value with a statistical summary of the known values.

2.1.1 1. Mean Imputation (μ)

This replaces NaNs with the arithmetic average of the column.

$$\hat{x}_{miss} = \mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

The Risk: The mean is highly sensitive to outliers. If you have a salary dataset where most people earn \$30k but one person earns \$10M, the mean will be artificially high. Filling missing values with this inflated mean introduces **bias** into your dataset. **Recommendation:** Use only when the data follows a **Normal (Gaussian) Distribution**.

2.1.2 2. Median Imputation (Q_2)

This replaces NaNs with the middle value of the sorted dataset.

$$\hat{x}_{miss} = \text{Median}(X)$$

The Benefit: The median is **robust** to outliers. It represents the "typical" value much better in skewed distributions. **Recommendation:** The default choice for continuous variables, especially skewed ones like Income or House Prices.

2.1.3 3. Mode Imputation

This replaces NaNs with the most frequent value. **Recommendation:** Use strictly for **Categorical Data** (e.g., missing "Color" or "City").

2.2 Multivariate Imputation: Leveraging Relationships

Simple imputation is "blind"—it ignores correlations. For example, if "Weight" is missing, simple imputation just guesses the average weight. But if we know the "Height" is very low, guessing the average weight is likely wrong. Multivariate methods use other features to make smarter guesses.

2.2.1 K-Nearest Neighbors (KNN) Imputation

This method assumes that "similar data points have similar values." To fill a missing value for a specific row (let's call it the Target), the algorithm: 1. Calculates the distance between the Target row and all other rows. 2. Selects the K rows that are "closest" (nearest neighbors). 3. Averages the values of those neighbors to fill the gap.

Numerical Example: How KNN Works

Consider a dataset with two features: **Age** (feature A) and **Income** (feature B). We have a missing Income for a 26-year-old.

$$D = \begin{bmatrix} \text{Age} & \text{Income} \\ 25 & 30k \\ 24 & 32k \\ 50 & 80k \\ 26 & \mathbf{?} \end{bmatrix}$$

Step 1: Calculate Distance. We measure the "closeness" based on the known feature (Age).

- Row 1: $|26 - 25| = 1$ (Very Close)
- Row 2: $|26 - 24| = 2$ (Close)
- Row 3: $|26 - 50| = 24$ (Far)

Step 2: Average Neighbors. The nearest neighbors are Row 1 and Row 2. We ignore Row 3 because it is statistically distant.

$$\text{Imputed Income} = \frac{30k + 32k}{2} = 31k$$

This method uses the local structure of the data rather than a global average.

2.2.2 Iterative Imputer (MICE)

This is the "Gold Standard" for tabular data. It treats the missing column as a **Target Variable (y)** and builds a Regression Model (like Linear Regression) using all other feature columns as inputs (X) to predict it. It does this iteratively for every column with missing data until the values stabilize.

Python Implementation Notes: Scikit-Learn provides `KNNImputer` and `IterativeImputer`. While computationally more expensive than simple imputation, they preserve the statistical correlations in your data significantly better.

3 Handling Outliers: Signal vs. Noise

An outlier is a data point that deviates significantly from other observations. But is it **Noise** (an error) or a **Signal** (a rare but real event)?

- **Noise:** A sensor reading of $500°C$ for room temperature. Action: Remove/Cap.
- **Signal:** A credit card transaction of \$10,000 when the average is \$50. Action: Keep (for fraud detection).

3.1 The IQR Method (Robust Detection)

The Interquartile Range (IQR) method is a robust statistical technique used to detect outliers. Unlike methods based on the Mean and Standard Deviation (Z-Score), the IQR method is based on the **Median** and is therefore not heavily skewed by the outliers it is trying to detect.

3.1.1 Step-by-Step Algorithm

To implement this, we follow a rigorous four-step process:

1. **Calculate Quartiles:**
   - Q_1 (25^{th} Percentile): The value below which 25% of the data falls.
   - Q_3 (75^{th} Percentile): The value below which 75% of the data falls.
2. **Compute IQR:** This represents the spread of the middle 50% of the data.
$$IQR = Q_3 - Q_1$$
3. **Define the "Fences":** We calculate theoretical boundaries beyond which data is considered anomalous.
$$\text{Lower Bound (LB)} = Q_1 - 1.5 \times IQR$$
$$\text{Upper Bound (UB)} = Q_3 + 1.5 \times IQR$$
4. **Decision Rule:** Any data point x_i is classified as an outlier if it falls outside these fences:
$$x_i < \text{LB} \quad \text{OR} \quad x_i > \text{UB}$$

3.1.2 Why 1.5?

The multiplier 1.5 is a statistical convention derived from the Gaussian distribution. Within these bounds (± 1.5IQR), we expect to find 99.3% of the data. Therefore, any point lying outside is statistically extremely rare (in the top 0.7% of extreme values).

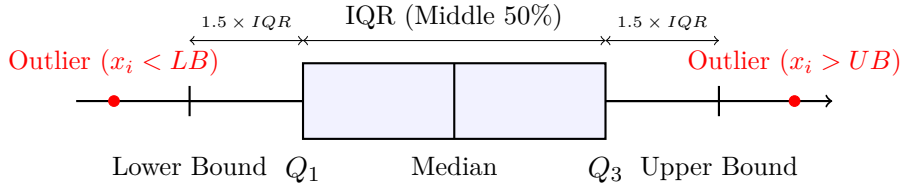

Figure 1: Visualizing the IQR Method (Box Plot). The "Box" contains the bulk of the data. The "Whiskers" extend to the calculated boundaries. Points outside these whiskers are flagged as outliers.

3.2 Treatment Strategies

Once detected, we have two main strategies: 1. **Trimming:** Deleting the rows. This is aggressive and reduces your dataset size. Use only if you have massive data and are sure the outliers are errors. 2. **Winsorizing (Capping):** Replacing the extreme values with the nearest boundary values (e.g., replacing any value $> UB$ with the value of UB). This suppresses the outlier's effect without losing the data point entirely.

4 Feature Scaling: The Geometry of Optimization

Many machine learning algorithms (Linear Regression, SVM, KNN, Neural Networks) calculate the **Distance** between points (Euclidean distance) or use **Gradient Descent** to find the minimum error.

If features have different scales (e.g., Age: 0-100 vs. Salary: 0-1,000,000), the algorithm will be biased. It will believe that a small change in Salary (1 unit = \$1) is insignificant compared to a change in Age (1 unit = 1 year), or vice versa depending on the math.

4.1 Normalization (Min-Max Scaling)

This squeezes the data into a fixed box, typically $[0, 1]$.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Effect: Preserves the shape of the original distribution but compresses it. **Use Case:** Neural Networks (which like inputs between 0 and 1) and Image Processing.

4.2 Standardization (Z-Score Scaling)

This centers the data around 0 and scales it to have a unit variance.

$$Z = \frac{X - \mu}{\sigma}$$

Effect: The resulting distribution has Mean $\mu = 0$ and Standard Deviation $\sigma = 1$. **Use Case:** The default for most algorithms (SVM, Logistic Regression). It makes the "loss landscape" symmetrical (like a bowl), allowing Gradient Descent to converge faster.

4.2.1 Mathematical Intuition: Why Gradients Fail Without Scaling

Why does scale matter? Consider the Gradient Descent update rule for a linear regression model $y = w_1x_1 + w_2x_2$:

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j}$$

The partial derivative of the cost function J with respect to a weight w_j is directly proportional to the magnitude of its associated feature x_j:

$$\frac{\partial J}{\partial w_j} \propto x_j \times \text{Error}$$

Scenario without Scaling:

- x_1 (Age) $\in [0, 100]$
- x_2 (Income) $\in [0, 100000]$

Since x_2 is 1000x larger than x_1, the gradient $\frac{\partial J}{\partial w_2}$ will be massively larger than $\frac{\partial J}{\partial w_1}$.

The Consequence: The optimizer will take gigantic, erratic steps in the w_2 direction (often overshooting the minimum) and microscopic, painfully slow steps in the w_1 direction. The path to the optimal solution becomes a chaotic zig-zag. Scaling forces the features to a similar range, ensuring $\frac{\partial J}{\partial w_1} \approx \frac{\partial J}{\partial w_2}$, which allows the optimizer to move in a straight, efficient line toward the minimum.

5 Categorical Encoding: Translating Text to Math

Machines understand numbers, not words. We must convert categories like "Red", "Blue", "Green" into numerical vectors.

5.1 The Ordinal Trap: Label Encoding

Label Encoding assigns a number to each category: Red=0, Green=1, Blue=2. **The Problem:** The model will mathematically interpret this as:

$$\text{Blue} > \text{Green} > \text{Red}$$

and

$$\text{Blue} - \text{Green} = \text{Red} \quad (2 - 1 = 0)$$

This mathematical relationship is nonsense for nominal data (colors have no order). Using Label Encoding for nominal data confuses the model and degrades performance. Use Label Encoding **only** if the data has a true rank (e.g., Low < Medium < High).

5.2 One-Hot Encoding

To avoid the ordinal trap, we create a new binary column for every category.

- Is_Red: [1, 0, 0]
- Is_Green: [0, 1, 0]
- Is_Blue: [0, 0, 1]

The Cost: This increases the number of features (dimensions). If a categorical column has 10,000 unique cities, One-Hot Encoding will add 10,000 columns to your dataset. This leads to the **Curse of Dimensionality**—data becomes sparse, and distance calculations become meaningless.

6 The Preprocessing Pipeline

To ensure reproducible and leak-free machine learning, we must chain these steps into a **Pipeline**.

Data Leakage Warning: Never fit your scaler or imputer on the *entire* dataset. You must calculate statistics (Mean, Median, Standard Deviation) **only** on the Training Set. Then, use those stored statistics to transform the Test Set. If you fit on the whole dataset, information from the Test Set "leaks" into the model during training, causing you to overestimate the model's performance.

Correct Pipeline Procedure

1. **Split** Data into Train (X_{train}) and Test (X_{test}).
2. **Fit** Imputer and Scaler on X_{train}.
3. **Transform** X_{train} using the fitted parameters.
4. **Transform** X_{test} using the SAME parameters (do not re-fit!).
5. **Train** Model on the processed X_{train}.

Python Implementation Snippet:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Define the sequence of steps
preprocessing_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Step 1: Fill missing values
    ('scaler', StandardScaler())                    # Step 2: Scale features
])

# Apply strictly to Train, then Test
X_train_processed = preprocessing_pipeline.fit_transform(X_train)
X_test_processed = preprocessing_pipeline.transform(X_test)
```